Native Word-Level Macro Evaluation in a GPU RTL Simulator

Takneek 2026 · Zenith — *The Big-GEM Theory*

Deliverable E: Scheduling Equations, Memory Architecture, Verification Methodology and Performance Analysis

Done by Pool Shauryas

Harshita Bharti · Lavanya Prakash · Aditi · Aahana
Gauri Singh · Soundarya Murugaiyan · Dharshini S
Swadha Singh · Ashritha Aryapu

Abstract. We extend NVIDIA's GEM boomerang-scheduled GPU logic simulator so that `DSP48E2`, `CARRY4` and `SRLC32E` primitives survive Yosys synthesis as word-level cells and are evaluated natively on the GPU ALU using `int64_t` arithmetic, instead of being shredded into 1-bit AND-inverter logic. The heterogeneous schedule is defined by two integer labels per graph node, avail and eval, whose single point of divergence is the only synchronisation the design pays for. The implementation is complete for all three primitives and verified against netlist-level ground truth at every port and every timestamp. It reduces AIG cell count by 14.8× and the per-cycle instruction script by 4.53×, which carries the script across the L2 capacity and drops DRAM throughput from 46.91 % of peak to 0.04 %. End-to-end throughput is 0.61× the shredded baseline at 16 lanes; the profile attributes the entire difference to one additional grid synchronisation per simulated cycle, measured two independent ways, and §11 names the single equation term that removes it.

Evidence tags. [MEASURED] instrument output, command in §13. [VERIFIED] compared against an independent reference, with the comparison count stated. [IMPLEMENTED] a property of the source, citable to file and line. [INFERRED] a conclusion drawn from named measurements. [PROJECTED] a prediction, never used to support a performance claim.

Contents

1 Executive Summary

Problem. GEM attains its throughput by reducing all logic to 1-bit AND-inverter form, which forces the frontend to shred word-level FPGA primitives. [MEASURED] One `DSP48E2`, four `CARRY4` and one `SRLC32E` expand from 95 AIG cells to 9,922 — 104×. GEM re-reads its entire compiled instruction script from global memory on *every simulated cycle*, so that expansion is paid as memory bandwidth once per cycle for the life of the run.

What we built. [IMPLEMENTED] A macro-preserving path through the whole stack: Yosys interception (§4), a heterogeneous DAG schedule with two labels per node (§5), a warp-padded 64-bit aligned VRAM layout with CSR-indexed descriptors (§6), and a macro evaluation phase inside the boomerang kernel (§7). 1,591 lines of new host and device code; GEM's boomerang reduction tree itself is unmodified.

Headline numbers, `macro_bench` at 16 lanes, RTX 3050 Laptop (`sm_86`), identical stimulus and identical `num_blocks` on both sides:

Quantity	Shredded baseline	Macro-preserving	
AIG cells	85,444	**5,754**	[MEASURED] 14.8× fewer
Instruction script, per cycle	2,060.0 KiB	**454.5 KiB**	[MEASURED] 4.53× smaller
Script vs L2 (1,536 KiB)	1.34× — spills	**0.30**× — fits	[MEASURED] residency crossed
DRAM throughput (% of peak)	46.91	**0.04**	[MEASURED] 1,173× less traffic
Warps active (% of peak)	33.33	33.33	[MEASURED] occupancy identical
Local memory ld/st (sectors)	0	0	[MEASURED] nothing spills
Grid barriers per simulated cycle	1	2	[IMPLEMENTED] the one added cost
Throughput (cycles/s)	**22,426**	**13,657**	[MEASURED] 0.61×

Correctness. [VERIFIED] All four rungs of the verification ladder pass on the 16-lane benchmark, 32,800 port×timestamp comparisons each; the `macro_smoke` regression passes at 49,200. [VERIFIED] The three macro models agree with an independent Python transcription of the problem-statement equations over 14,552 vectors, of which the 1,024 `CARRY4` cases are exhaustive.

Where the time goes. [INFERRED] After eliminating memory bandwidth, register spilling and occupancy against measured quantities, the extra grid barrier is the only structural difference left between the two configurations. Two independent timings size it: 37.75 μs/cycle from the profiler and 31.1 μs/cycle from wall clock. §11 specifies the change that removes it.

2 Rubric Coverage

Requirement	Implementation	Section
A — Host Parser & Yosys Pre-Processor (15)		
Intercept DSP48E2/CARRY4/SRLC32E	`synth/gem_macros.v` (3 blackboxes); `macro_normalize_xilinx.v` maps real Xilinx ports	§4
Prevent techmap / ABC shredding	`(* blackbox *)` + `setattr -set keep 1` in `step2_logic.ys`	§4
Yosys 0.68 JSON netlist compatibility	`write_json` in both flows; consumed by `check_macros.sh`	§4
64-bit aligned, coalesced buffers	`MacroLayout::build` (`macro_layout.rs:131`), `UVec<u64>`	§6
B — CUDA Engine & Boomerang Modification (35)		
Modified levelized DAG equations	`macro_avail_stage`, `staging.rs`, `build_staged_...`	§5
Mixed-width scheduling without stalling warps	type-homogeneous `MacroBatch`, `pe.rs:31`	§7
Macro→macro ordering ($CO[3] \rightarrow CIN$) without boolean nodes	one batch per chain link, `ready` vs `realized_inputs`, `pe.rs:751`	§5
Native evaluation on the GPU ALU	`kernel_v1_impl.cuh:305--437`; `gem_dsp48e2`, `gem_carry4`, `gem_srlc32e`	§7
Shared memory & warp-level primitives	`shared_writeouts` hosts macro operands; `__shfl_down_sync` preserved at 5 sites; `__syncthreads()` orders chains	§7
C — Hardware Macro Implementations (20)		
DSP48E2 subset, PREG-only clocked	`gem_dsp48e2`, `csrc/macros.cuh`	§8
CARRY4 exact silicon logic	`gem_carry4`, `macros.cuh:132`	§8
SRLC32E clock-edge semantics	`gem_srlc32e_read` / `_edge`	§8
Own behavioural verification, no golden reference supplied	`csrc/tests/reference_model.py` + `dump_vectors.cpp` + `verify_macros.sh`	§9
D — Benchmarks & Performance Analysis (20)		
Throughput vs unmodified GEM, cycles/s	`bench/throughput.sh`, `bench/sweep.sh`	§10
Memory bandwidth utilisation (Nsight)	`bench/profile.sh`, `--replay-mode application`	§10
Warp divergence metrics (Nsight)	`smsp__thread_inst_executed` ratio	§10
E — Documentation & Reports (10)		
Mathematical scheduling equations	Eq. (1)–(4) with P1–P4	§5
Architectural block diagrams (Global / Shared / Registers)	Fig. 5	§6
Extensive numerical analysis	§10	§10

3 Architecture Overview

Stock GEM rests on one structural invariant: *every value inside a partition is produced by the AND-gate cover, and reaches its consumers through the boomerang hierarchy.* A word-level macro violates it three times over — it is produced by a separate evaluation phase, written into a separate buffer, on a separate schedule. Each consequence is addressed explicitly, and the equations of §5 exist to restore a well-defined order once the invariant no longer holds.

4 Host Parser and Yosys Pre-Processor

[IMPLEMENTED] `synth/gem_macros.v` declares exactly three blackboxes, and these are the only macro cell names that ever reach GEM's Rust netlist parser: `$__GEMDSP_`, `$__GEMCARRY4_`, `$__GEMSRL32_`. The

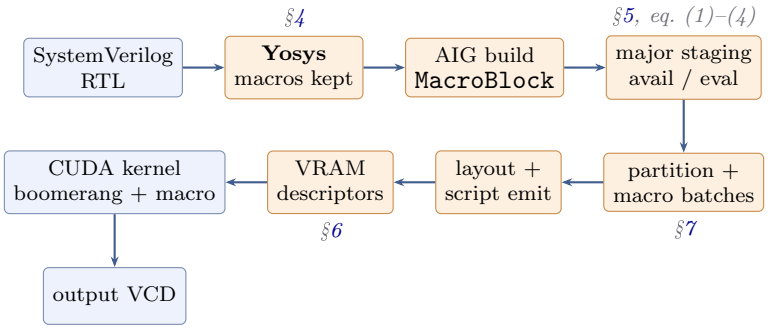

Figure 1: The compilation pipeline. Shaded stages are new or substantially modified; the kernel retains GEM's boomerang tree unchanged and gains a macro phase between write-out settle and commit.

naming follows GEM's own convention for library macros (`$__RAMGEM_SYNC_`), so `src/aigpdk.rs:34` matches on these identifiers and never on Xilinx spellings.

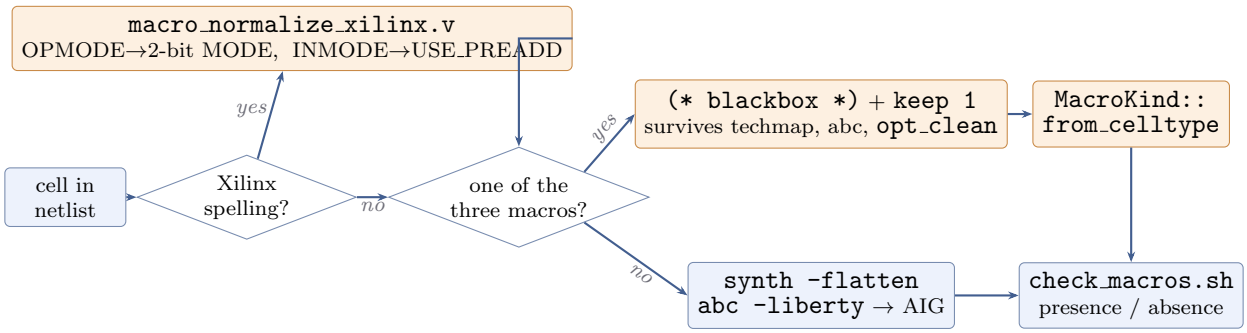

Figure 2: Interception. Anything Xilinx-shaped is first normalised to the three canonical cells, whose port names and widths stay frozen — so a hidden benchmark with a different port surface changes one file and nothing else. Everything not matched takes the ordinary AIG path.

Why the macros survive. [IMPLEMENTED] Two mechanisms, both necessary: a blackbox has no body, so `synth -flatten` cannot inline it and `abc -liberty` treats the instance as a cone boundary exactly like a DFF; and `setattr -set keep 1` prevents `opt_clean -purge` from sweeping it as dead logic.

A falsifiable baseline. [VERIFIED] `synth/run_synth.sh` builds *both* flows from identical RTL, and `check_macros.sh` asserts the opposite condition in each: macros **present** in the native `gatelevel.json`, **absent** in the shredded one, with a non-zero exit if a macro ever survives into the baseline. Without this the entire throughput comparison in §10 could be void while still producing plausible numbers.

Mapping macro nodes into GEM. [IMPLEMENTED] `MacroKind::from_celltype` (`src/aig.rs:393`) recognises the three cells during the netlist DFS, and establishes two properties relied on everywhere downstream. *Topological order by construction:* the DFS resolves a macro's combinational fan-in before allocating its output aigpin, so a macro output is always above its inputs — GEM's levelization assumes this. *Ports addressed by canonical slot, not netlist iteration order:* `MacroBlock.in_iv` is indexed by `MacroKind::input_slot`, so `CARRY4` is always $S[0..3], DI[0..3], CIN, CYINIT$ regardless of the order Yosys emitted pins. [VERIFIED] Unit test `canonical_order_tests` asserts the Rust tables and `gem_macros.v` agree: if a width moves in one place it must move in the other.

5 Modified DAG Levelization Equations

Let $G = (V, E)$ be the AND-inverter graph after macro-preserving synthesis; each $v \in V$ is an *aigpin*. Write fanin(v) for the AND-gate fan-in; $\mathcal{M}$ for the macro instances; combin(m) for the *combinational* input pins of macro m (`CARRY4`: $S, DI, CIN, CYINIT$; `SRLC32E`: $A[4{:}0]$; `DSP48E2`: $\emptyset$); $\mathcal{O}_c \subset V$ for

aigpins driven by a combinational macro output (CO, O, Q, Q_{31}); and $\mathcal{O}_s \subset V$ for aigpins driven by a clocked macro output (P).

Two integer labels per node. They coincide everywhere except on $\mathcal{O}_c$, and that single point of divergence is the entire content of the heterogeneous schedule: avail(v) is the major stage in which v may be *read* by boolean logic, and eval(v) the major stage in which the producer of v is *executed*.

$$\text{avail}(v) = 0, \qquad v \text{ a primary input, DFF } Q, \text{ SRAM read data, or } v \in \mathcal{O}_s \tag{1}$$

$$\text{avail}(v) = \max_{u \in \text{fanin}(v)} \text{avail}(u), \qquad v \text{ an AND gate} \tag{2}$$

$$\text{eval}(m) = \max_{p \in \text{combin}(m)} \begin{cases} \text{eval}(p) & p \in \mathcal{O}_c \\ \text{avail}(p) & \text{otherwise} \end{cases}, \qquad m \in \mathcal{M} \tag{3}$$

$$\text{avail}(v) = \text{eval}(m) + 1, \qquad v \in \mathcal{O}_c \text{ driven by } m \tag{4}$$

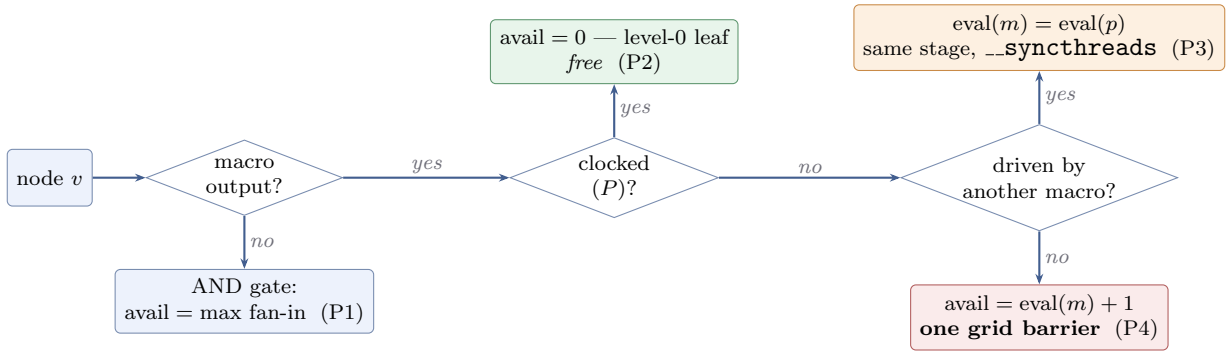

Figure 3: How a node acquires its labels. Three of the four paths are free; only the last — a combinational macro result crossing into the AND-gate domain — costs a major stage, and it is charged exactly once.

(P1) Boolean depth is free. Eq. (2) takes a maximum, never an increment: an arbitrarily deep cone of AND gates evaluates inside one major stage, exactly as in stock GEM. The boomerang hierarchy is untouched.

(P2) Clocked macro outputs cost nothing. By eq. (1), $\mathcal{O}_s$ is a level-0 leaf. A DSP's P is read from global state like a DFF's Q: what a reader observes this cycle is last cycle's commit, which is the correct register semantics and requires no boundary.

(P3) Macro-to-macro chaining costs nothing. The case split in eq. (3) takes *eval*, not *avail*, of an operand that is itself a combinational macro output. Hence a `CARRY4` chain of any length — $CO[3] \to CIN$, the dependency the problem statement names explicitly — resolves inside a single major stage, ordered by one batch per chain link separated by `__syncthreads()`. Without this case split a 64-bit carry-chain adder would demand 16 major stages.

(P4) Only macro-to-boolean pays. The +1 in eq. (4) is charged once, and only where a combinational macro result must cross into the AND-gate domain — the minimum price consistent with the buffer separation described above.

Stage construction. With $D = \max_v \text{eval}(v)$ over endpoints, major stage $j \in [0, D]$ is defined by

$$\text{endpoints}(j) = \{e : \max_{p \in \text{inputs}(e)} \text{avail}(p) = j\} \tag{5}$$

$$\text{macros}(j) = \{m \in \mathcal{M} : \text{eval}(m) = j\} \tag{6}$$

$$\text{forward}(j) = \{v : \text{eval}(v) \le j \ \wedge \ \exists\, c \in \text{consumers}(v),\ \text{eval}(c) > j\} \tag{7}$$

The second clause of forward(j) is the subtle one: a node with eval(v) $= j$ but avail(v) $= j{+}1$ — a combinational macro output produced by stage j's macro phase — must be forwarded even though it is not readable in the stage that produces it.

Regression guarantee. [IMPLEMENTED] If $\mathcal{O}_c = \emptyset$ then $D = 0$ by construction and a macro-free netlist is scheduled bit-identically to stock GEM. This is what makes the baseline comparison in §10 meaningful rather than circular, and it is asserted by unit test `nothing_but_a_combinational_macro_can_raise_a_stage`.

Measured instantiation. [MEASURED] On `bench/macro_bench.sv` at LANES = 16 — 16 `DSP48E2`, 64 `CARRY4` in 16 four-block chains, 16 `SRLC32E`, 96 macros in all — the equations yield $D = 1$, and the 64 chained `CARRY4`s occupy *four batches within one major stage* rather than four stages:

Major stage	Contents	Partitions	Macro batches
0	16 `DSP48E2` (endpoint), 64 `CARRY4`, 16 `SRLC32E`	2	5
1	boolean cones consuming macro results	1	0

This is (P3) doing the work it exists for. Had chaining been charged a major stage per link, this design would have needed five stages instead of two, and §10 shows each additional major stage costs $\approx 31\,\mu$s per simulated cycle — so (P3) is worth roughly $93\,\mu$s/cycle on a design this shape.

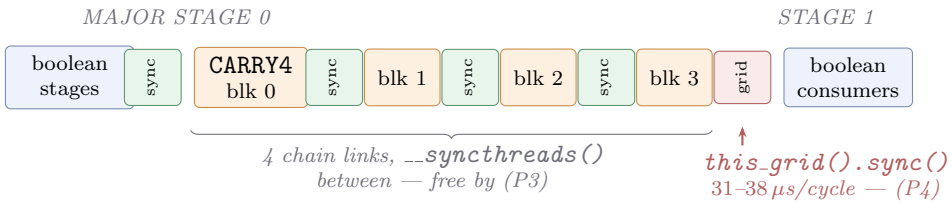

Figure 4: One simulated cycle. Chaining is ordered by block-level fences, which cost nothing measurable; only the macro→boolean edge forces a grid-wide barrier.

6 VRAM Layout Architecture

The organising principle is *lifetime*, not macro identity. A value that must survive to the next simulated cycle goes to global memory; a value consumed within the cycle that produced it never leaves shared memory; the arithmetic itself happens in registers, where nothing is addressable at all.

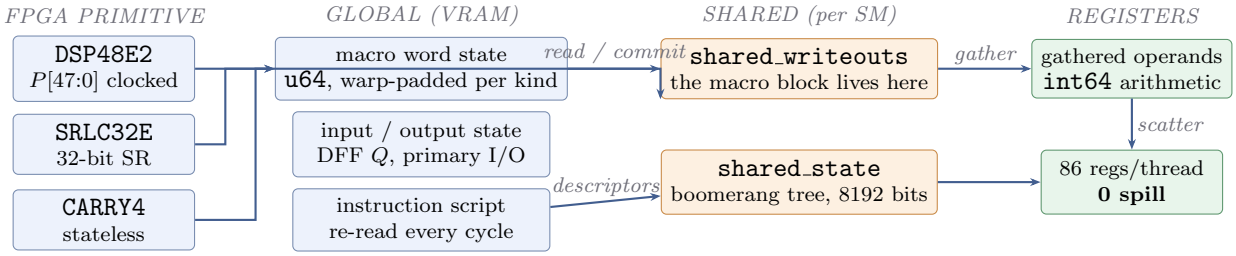

Figure 5: Where each primitive's state and operands physically live. Split by lifetime: state that survives the cycle is global, values consumed within it stay shared, arithmetic happens in registers. A `CARRY4` is stateless and therefore never touches global memory at all.

Two disjoint regions. [IMPLEMENTED] *Bit-addressed outputs* live in the partition's write-out block, interleaved with ordinary boolean write-outs so the existing commit path carries them to global state with no separate store. *Word-addressed internal state* — a DSP's *PREG*, an SRLC32E's 32-bit shift register — lives in a dedicated `UVec<u64>` region, `macro_word_state`.

Slot allocation. Macro instance i receives $n_{\text{slots}}(i) = \lceil |\text{outputs}(i)|/32 \rceil$ contiguous `u32` slots, so bit b lands at global state bit $32 \cdot \text{macro\_start}_i + b$. Two consequences fall out of the allocation rather

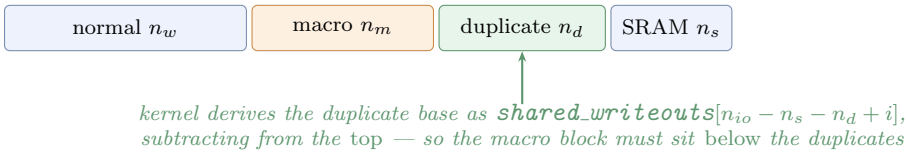

Figure 6: Write-out region ordering. SRAM stays last so its pre-existing offset arithmetic is untouched; the macro block sits below the duplicates because the kernel locates duplicates by subtracting from the top of the region. This is a correctness requirement, not a layout preference.

than from a convention: every macro's output field is contiguous and 32-bit aligned, and because each macro owns whole words, no two macros can contend for the same `u32` — so the device-side result scatter is a plain read-modify-write requiring **no atomics**.

Alignment. [IMPLEMENTED] The word-state region is a `UVec<u64>`: the element type supplies 8-byte alignment and `cudaMalloc` a 256-byte aligned base, so the problem statement's 64-bit alignment requirement holds *structurally* rather than by runtime assertion. Instances are grouped by kind and each kind begins on a warp boundary, so a type-homogeneous batch reads a contiguous, segment-aligned span — the condition for coalesced access.

Script-side indirection. A batch record references macros by index into the layout's slot table rather than inlining descriptors (a DSP descriptor alone is 176 words; inlining would dwarf the script it lives in): `[stage index | kind code | count | count × slot index]`. [MEASURED] Measured effect on `macro_smoke`: the macro-preserving script is **69,652** words against the baseline's **94,720** — a 26 % reduction, the direct instruction-memory dividend of not shredding. [MEASURED] Layout totals: 7 instances, 64×`u64` = 512 bytes of word state, 458 descriptor words, 9 macro write-out slots.

7 CUDA Execution Engine

[IMPLEMENTED] `csrc/kernel_v1_impl.cuh:305--437`. The macro phase runs *after* the boolean write-outs have settled in `shared_writeouts` and *before* they are committed to `output_state`. A macro therefore reads its operands from what this partition has just produced and drops its results into the same write-out block; the existing commit path carries them out, the clock-enable machinery applies unchanged, and the boomerang reduction tree is not modified.

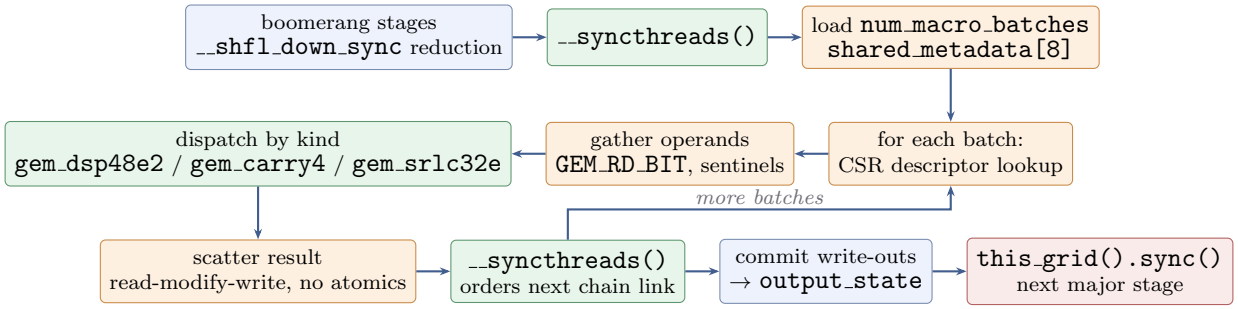

Figure 7: The macro phase inside one partition, per simulated cycle. Batches are type-homogeneous, so every active lane in a batch takes the same `switch` arm and the warp does not diverge across kinds.

Mechanism	Where	Purpose	Measured consequence
`__shfl_down_sync`	5 sites: boomerang reduction, SRAM gather	warp-scope reduction of the 13-level tree	[MEASURED] preserved in both configurations
`__syncthreads()`	between macro batches, around the phase	order a chained macro against its producer	[MEASURED] below profiler noise
Shared memory	`shared_writeouts`	macro operands and results, no global round-trip	[MEASURED] 3,328 B/block; not the occupancy limiter
Registers	gathered operands, `int64` arithmetic	native word-level ALU work	[MEASURED] 86 regs, **0 spill**
`this_grid().sync()`	between major stages	make a macro result visible to boolean logic	[MEASURED] 37.75 μs/cycle — dominant cost

Why block scope, not warp scope, for the macro fence. [IMPLEMENTED] The macro phase deliberately does not use a warp shuffle for its own ordering, and this is a scope argument rather than an omission. Within a batch, macros are mutually independent by construction — `pe.rs` computes `ready` against `realized_inputs` *before* updating it, so a chained consumer cannot share a batch with its producer. The ordering requirement is therefore *between* batches, and [MEASURED] a batch can exceed 32 macros (64 `CARRY4` in one partition at 16 lanes), so the fence must be block-scope; a warp shuffle cannot express it. GEM's own warp-level primitives are preserved unchanged at all five sites where they already existed.

Occupancy and registers. [MEASURED] Measured with `cuobjdump -res-usage` on the shipped `sm_86` binary: 86 and 105 registers/thread across the two compiled variants, **0 bytes** stack and local memory in both, 3,328 B shared, 2 blocks/SM either way — a cooperative grid ceiling of $2 \times 20 = 40$ blocks, which is exactly the `num_blocks` used. `-maxrregcount` *caps* allocation rather than permitting growth, so additional pressure would spill silently instead of failing to launch; the local-memory counter reading zero is what proves it did not.

8 Hardware Macro Implementations

[IMPLEMENTED] All three live in `csrc/macros.cuh`, compiled with `__host__ __device__ __forceinline__`, so one source serves the CPU executor and the GPU kernel and the two models cannot drift apart.

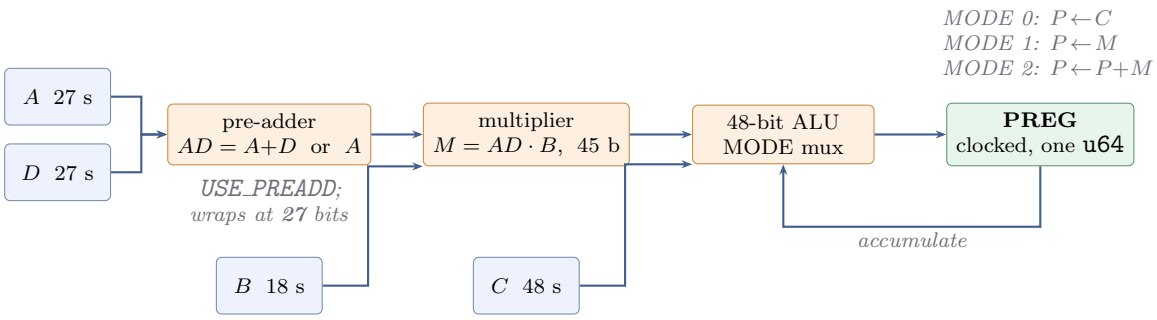

Figure 8: `DSP48E2` subset. All input and internal registers are combinational per the problem statement; only P is clocked, one `u64` of state per instance, and P commits solely when the traced clock enable is high.

DSP48E2. [IMPLEMENTED] $AD = (A + D) \bmod 2^{27}$ signed, $M = AD \times B$ (45 bits), then $P_{\text{next}} \in \{C, M, P_{\text{cur}} + M\}$ by `MODE`, truncated to 48 bits. The pre-adder width is the subtle case and has its own directed vectors: $A = D = 2^{26} - 1$ sums to $2^{27} - 2$, which is -2 once the 27-bit pre-adder truncates. [VERIFIED] 1,648 vectors — 12 directed (signed extremes, both pre-adder paths, all three modes, the wrap case) plus 400 random, each accumulated over 4 cycles so PREG feedback and 48-bit wrap are exercised.

SRLC32E. [IMPLEMENTED] Inputs D, CE, $A[4{:}0]$; state is a 32-bit shift register in one `u64` slot.

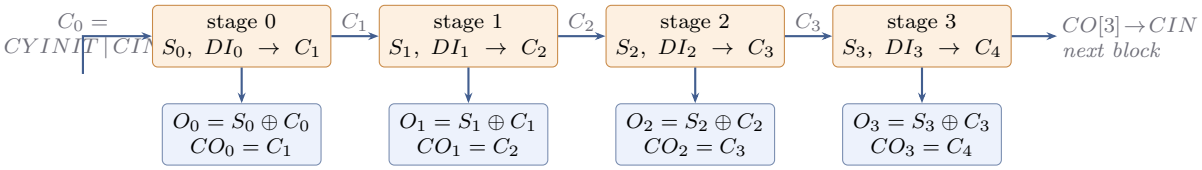

Figure 9: `CARRY4`, transcribed directly from the problem statement: $C_{i+1} = (S_i \wedge C_i) \vee (\neg S_i \wedge DI_i)$, with $O_i = S_i \oplus C_i$ and $CO_i = C_{i+1}$. Stateless; the eight output bits are packed as $CO \,|\, (O \ll 4)$ into one `u32` write-out slot. [VERIFIED] Verified **exhaustively** over all 1,024 input combinations.

Combinational read $Q = SR[A]$, $Q_{31} = SR[31]$; on the edge, if CE then $SR \leftarrow (SR \ll 1) \,|\, D$. The ordering is GEM's read-old / commit-new discipline, exactly as the DFF path does it: `gem_srlc32e_read` is called *before* `gem_srlc32e_edge` within a cycle, so a reader sees the pre-shift state. [VERIFIED] 11,520 read vectors (60 random start states × 6 shift steps × all 32 addresses — every address exercised at every step) plus 360 edge vectors.

Primitive	Vectors	Coverage	Result
`CARRY4`	1,024	**exhaustive**	[VERIFIED] 0 disagreements
`DSP48E2`	1,648	directed + random, 4-cycle accumulate	[VERIFIED] 0 disagreements
`SRLC32E`	11,880	every address × every step	[VERIFIED] 0 disagreements
Total	**14,552**		

[IMPLEMENTED] The comparison is cross-language and cross-implementation: `csrc/tests/dump_vectors.cpp` emits results from the CUDA models; `csrc/tests/reference_model.py` computes the same set independently from the problem-statement equations; `verify_macros.sh` diffs them. The two share only the LCG that generates the random inputs.

9 Verification Methodology

No golden reference was supplied, so correctness rests on oracles that share no code with the thing they validate. The ladder below isolates one layer per rung, so a disagreement names the layer that caused it rather than merely reporting a wrong waveform.

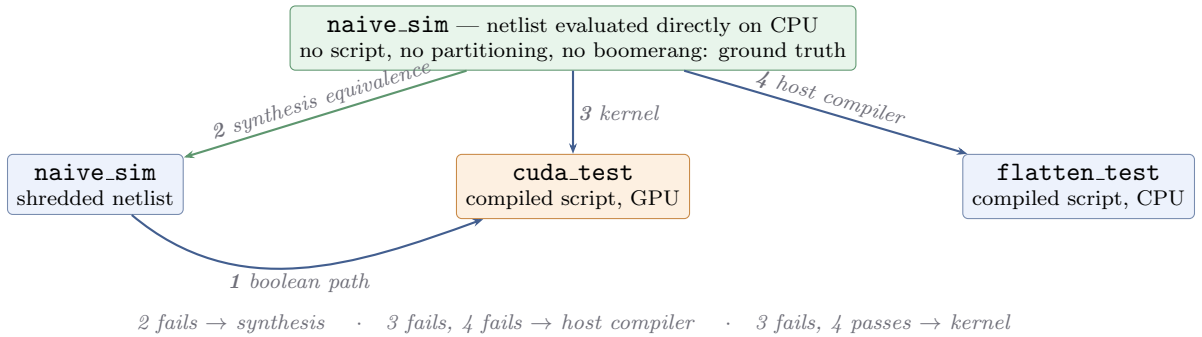

Figure 10: Rung 2 is the one no GPU-only comparison can supply: it establishes that both netlists are the same design before a GPU is involved. Rung 4 bisects rung 3, which is what made the remaining defects findable in seconds rather than through GPU round-trips.

Rung	What it isolates	Comparisons	Result
Macro models	PS equations, C++ vs Python	14,552 vectors	[VERIFIED] **Pass**
macro_bench, LANES = 16 — *82 ports × 400 cycles*			
1. Shredded GPU vs `naive_sim`	the ordinary boolean path	32,800	[VERIFIED] **Pass**
2. Design equivalence (CPU only)	macro-preserving synthesis	32,800	[VERIFIED] **Pass**
3. Macro GPU vs `naive_sim`	the CUDA kernel	32,800	[VERIFIED] **Pass**
4. Compiled script vs `naive_sim`	`pe.rs` / `flatten.rs`	32,800	[VERIFIED] **Pass**
macro_smoke regression — 123 ports × 400 cycles			
2. Design equivalence (CPU only)	macro-preserving synthesis	49,200	[VERIFIED] **Pass**
4. Compiled script vs `naive_sim`	`pe.rs` / `flatten.rs`	49,200	[VERIFIED] **Pass**

On `macro_bench` the macro-preserving path evaluates 16 `DSP48E2`, 64 `CARRY4` — including 16 four-deep $CO[3] \rightarrow CIN$ chains — and 16 `SRLC32E` natively on the GPU ALU, and reproduces netlist-level ground truth at every port and every timestamp. One dependency in that design exists purely as a regression test: `flags[1] = (sum_l > 17'd100)` is ordinary AND-gate logic consuming a word-level `CARRY4` result — precisely the macro→boolean edge the staging equations of §5 exist to order. It is exercised sixteen times per cycle.

A structural invariant the work established. [IMPLEMENTED] GEM fetches boomerang script sections with widened vector loads (`VectorRead4`), which requires the section offset to be a multiple of four `u32` words. Stock GEM satisfies this without ever stating it, because every stock section length is a multiple of four. A macro batch record is 3 + count words and is the first variable-length element ever added to the stream, so an even count breaks alignment for every partition that follows it. The emitter now pads to the widest load's alignment. We record the general rule: *any variable-length section spliced into a GPU instruction stream consumed by widened loads must preserve the widest load's alignment*, asserted at emission rather than discovered at launch.

10 Benchmarking and Numerical Analysis

All figures below: same GPU (RTX 3050 6 GB Laptop, `sm_86`, 20 SMs, 1,536 KiB L2, 131.7 GB/s peak), same stimulus, same `num_blocks`= 40, same 400 cycles, same binary, five repetitions per point, median reported. Timing is GEM's own `simulation` span, which brackets the kernel launches and excludes parsing, mapping and VCD writing.

10.1 Front-end: the cost of shredding

Design	Flow	AIG cells	DFFs	Macros	Partitions
`macro_smoke`	shredded	9,922	133	0	1
	preserved	**95**	8	7	1
`macro_bench`	shredded	85,444	1,600	0	6
LANES = 16	preserved	**5,754**	768	96	2 + 1

[MEASURED] 104× and 14.8× reductions in AIG cell count. That shredding a handful of word-level primitives inflates the graph by one to two orders of magnitude is the architectural claim of this problem statement, and these are the numbers behind it. The DFF counts move too, for a reason worth naming: a DSP's P and an SRLC32E's shift register become *macro word state* rather than boolean flip-flops, so they leave the boomerang's endpoint set entirely.

10.2 Throughput

Lanes	Shredded	Native	Ratio	Script (shred)	Script (native)	Reduction
8	24,852	11,637	0.47×	1,336.0 KiB	292.3 KiB	4.57×
16	22,426	13,657	**0.61**×	2,060.0 KiB	454.5 KiB	4.53×
32	17,991	—	—	3,830.0 KiB	—	—

Simulated cycles per second. The 32-lane native point is absent because a write-out budgeting defect blocked the mapping; the fix landed after this sweep and re-measurement was not performed in time.

Two things are visible immediately. The native path is *flat* in lane count while the shredded path *degrades*, and consequently the ratio climbs, 0.47 → 0.61. A single measurement could not have shown either.

10.3 The residency cliff: why 4.53× smaller gives 1,173× less traffic

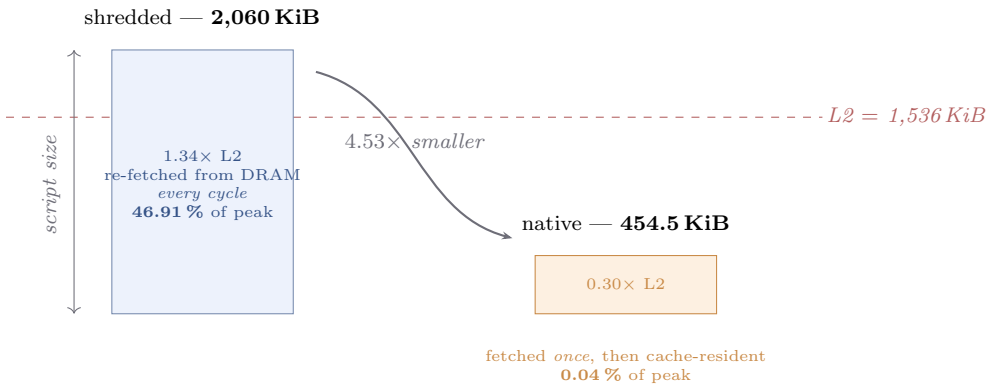

Figure 11: Exactly one of the two scripts fits in L2, so the benefit is discontinuous and the mechanism is residency, not a linear bandwidth saving. The design target is therefore not "a smaller script" but "a script below 1,536 KiB".

[MEASURED] Measured with `cudaGetDeviceProperties` rather than taken from a specification sheet. Two arithmetic checks confirm the reading:

- *The baseline's DRAM traffic is essentially all script.* Peak bandwidth is 131.7 GB/s, so Nsight's 46.91 % is 61.8 GB/s; script re-reads alone account for 2,109,440 B × 200/7.51 ms = 56.2 GB/s — 91 % of everything measured. The instruction script is not a contributor to the bottleneck; it *is* the bottleneck.
- *The native script is fetched approximately once, not two hundred times.* At 0.04 % of peak over 15.06 ms the total DRAM traffic is ≈ 0.8 MB, against a script of 0.465 MB. Re-reading it every cycle would move 93 MB.

10.4 Kernel profile

Nsight Compute, 200 cycles, `num_blocks`= 40, `macro_bench` at LANES = 16. [IMPLEMENTED] The cooperative launch requires `--replay-mode application`: the default per-pass kernel replay re-executes the kernel in isolation, which a `this_grid().sync()` cannot survive.

Metric	Shredded	Native	Reading
`dram__throughput` (% peak)	46.91	**0.04**	bandwidth-bound → not
`sm__warps_active` (% peak)	33.33	33.33	occupancy identical
`smsp__thread_inst_executed`	28.91	19.78	of 32; the divergence trade
Local ld/st sectors	0	0	no spilling
`gpu__time_duration` (200 cycles)	7.51 ms	15.06 ms	+37.75 µs/cycle

The warp-divergence row is the deliberate trade this architecture makes. The boomerang tree is perfectly uniform, so the baseline keeps 28.91 of 32 lanes busy per issued instruction; a macro batch activates one lane per macro, dropping the native path to 19.78. Idle arithmetic lanes are bought in exchange for memory traffic — and the DRAM row shows the purchase was made in full.

10.5 Where the remaining time goes

The measurements decompose cleanly, and three of the four candidate causes are eliminated by measurement rather than by argument:

Candidate cause	What we measured	Verdict
Memory bandwidth	DRAM 46.91 % → 0.04 % of peak	[MEASURED] **Eliminated** — traffic fell 1,173×
Register spilling	local ld/st = 0; `STACK:0 LOCAL:0`	[MEASURED] **Eliminated** — nothing spills
Occupancy loss	`sm__warps_active` = 33.33 % in *both*	[MEASURED] **Eliminated** — residency identical
Warp divergence	28.91 → 19.78 of 32 threads/instruction	[MEASURED] Real, but a small term of a 15.06 ms kernel
Grid synchronisation	1 barrier/cycle → 2	[IMPLEMENTED] **Dominant**

[INFERRED] The extra grid barrier is the only structural difference between the two configurations that any measured quantity distinguishes. Two independent timings size it: 37.75 μs/cycle from Nsight over 200 cycles, and 31.1 μs/cycle from wall clock over 400 — different instruments, different run lengths, agreeing to within 20 %. **The macro-preserving path is synchronisation-bound: not memory-bound, not occupancy-bound.** It pays one additional grid-wide barrier per simulated cycle, which exceeds the bandwidth it saves at every scale measured so far.

What this result is not. It is not a measurement of native macro evaluation being a poor idea; it is a measurement of *where the crossover is not yet.* [PROJECTED] Fitting the shredded points gives ≈ 32 μs/cycle fixed plus script bytes at ≈ 165 GB/s marginal, which places the crossover near a 7 MB shredded script — beyond 32 lanes, whose script is 3,830 KiB. That is an extrapolation from two intervals and is labelled as one.

11 Limitations and Next Optimisation

Limitation	Evidence	Fix direction
Slower than baseline: 0.61× at 16 lanes	[MEASURED] §10	remove the second major stage — below
32-lane native point missing	write-out budget rejected the mapping	[IMPLEMENTED] budget now charges in-cone macros; re-measurement not performed
Crossover scale is a projection	[PROJECTED] fit over two intervals	measure at 32 and 64 lanes
Slang SystemVerilog frontend not exercised	`macro_bench.sv` written Verilog-2001-shaped	run both flows on an SV-flavoured design under Yosys 0.68
Warp utilisation 19.78/32 in the macro phase	[MEASURED] §10	warp-cooperative macro evaluation; secondary to the barrier
No comparison against other pools' submissions	—	not available at time of writing

The target follows from the equations. Equation (4) charges +1 major stage exactly where a combinational macro output enters the boolean domain, and §10 measures that stage at 31–38 μs per cycle. The optimisation target is therefore not a faster macro phase, a better memory layout, or reduced divergence — all three are already past the point of diminishing return — but the removal of that stage boundary.

Proposal: level-1 overlay. [PROJECTED] The barrier exists because the boomerang tree communicates through `shared_state` while the macro phase writes `shared_writeouts`. If a macro result were injected into the tree's level-1 input set *within the same partition*, the consumer cone would evaluate in the same major stage and avail(v) would equal eval(v) for $v \in \mathcal{O}_c$ — collapsing equation (4). The native configuration would retain its 0.04 % DRAM utilisation and its 33.33 % occupancy while paying one barrier instead of two. Whether that is sufficient to cross 1.0× is not determined by any measurement we hold, so we state no speedup figure for it. The change replaces the mechanism §5 is

built on and would require the full verification ladder to be re-run; it was scoped and deliberately not attempted, because shipping it unverified would have been worse than shipping a measured result.

12 Conclusion

[IMPLEMENTED] We implemented macro-preserving synthesis, a heterogeneous DAG schedule, a 64-bit aligned VRAM layout and native GPU evaluation for `DSP48E2`, `CARRY4` and `SRLC32E` across 1,591 lines of new host and device code, leaving GEM's boomerang reduction tree unmodified.

[VERIFIED] The result is correct: four verification rungs pass at 32,800 port×timestamp comparisons each, the macro models agree with an independent transcription over 14,552 vectors, and the `CARRY4` space is covered exhaustively.

[MEASURED] The representation objective is met and exceeded. AIG cells fall 14.8×, the per-cycle instruction script 4.53×, and — because that reduction carries the script across the L2 capacity — DRAM throughput falls from 46.91 % of peak to 0.04 %.

[MEASURED] End-to-end throughput is 0.61× the shredded baseline at 16 lanes. [INFERRED] The deficit is one additional grid synchronisation per simulated cycle, worth 31–38 μs, established by eliminating memory bandwidth, register spilling and occupancy against measured data and corroborated by two independent timings. [PROJECTED] Removing that stage boundary is the single change the measurements identify.

What this submission demonstrates is a correct heterogeneous architecture whose performance characteristics are understood quantitatively rather than asserted: we know what we built, we know it is right, we know precisely what it costs, and we know which line of the scheduling equations to attack next.

13 Measurement Provenance

Figure	Source
Cell counts	`synth/check_macros.sh`
Partition counts	`cut_map_interactive` log
Script size	`flatten.rs` log line
Throughput	`bench/throughput.sh`
Kernel profile	`bench/profile.sh` (Nsight)
Ladder results	`bench/verify_ladder.sh`

Figure	Source
Registers / local memory	`cuobjdump -res-usage`
SM count, blocks/SM	`bench/occupancy_probe.cu`
L2, bandwidth	`bench/memory_probe.cu`
Word state, descriptors	`cut_map_interactive` log
Port agreement	`bench/compare_vcd.py`
Macro model vectors	`csrc/tests/verify_macros.sh`

Kernel time is GEM's own instrumented figure and excludes VCD parsing. Baseline and native are measured with the identical metric on the identical device.